Atlas 题库已完成题目汇总（[done] 共 115 题）

> 统计口径：knowledge-base 全部 Q&A 册中状态标记为 [done] 的题目；每题摘录答案首段结论（各册写作规范要求首段先给结论）。

04-language（20 题）

04-language/C++/01-语言基础.md（3 题）

Q9：命名空间如何区分同名实体？

同名实体位于不同命名空间时，用限定名指定目标实体。比如 Android 与厂商模块都定义 `init::start()`，可分别写成 `android::init::start()` 和 `vendor::init::start()`。

Q26：std::cerr 与 std::endl 如何刷新输出？

`std::cerr` 是标准错误流，默认设置 `unitbuf`，输出操作后会刷新关联的流缓冲区。`std::endl` 写入换行符并显式刷新所作用的流。`<<` 负责将内容插入流。

Q27：std::cout 输出后程序阻塞，如何确保内容已刷新？

例如：

04-language/C++/02-编译与链接.md（7 题）

Q2：多文件的 C++ 工程为什么需要头文件？声明在编译和链接阶段各起什么作用？

C++ 通常按翻译单元独立编译。编译器处理一个 `.cpp` 时不会读取其他 `.cpp`，而是分别生成目标文件，再由链接器解析跨文件的符号引用。头文件提供可共享的声明，使调用方能在编译期检查调用，链接器随后将引用与定义关联。

Q3：头文件中的 #pragma once 是什么？

`#pragma once` 是放在头文件中的编译器指令，告诉支持它的编译器：处理同一个翻译单元时，该头文件只需包含一次，避免其中的类型等被重复定义。

Q5：#include "util.h" 与 #include <util.h> 的搜索范围有何不同，项目头文件应该用哪种？

两种写法的主要区别是搜索规则：具体搜索位置由实现和构建选项决定。常见编译器会让 `"..."` 先搜索发起包含的文件所在目录，再搜索常规头文件路径。`<...>` 使用常规头文件搜索路径。通常用尖括号包含标准库或外部依赖头文件，用引号包含项目头文件。

Q6：头文件中的 #ifndef、#define、#endif 有何作用？#pragma once 能否替代？

这三行构成头文件保护宏（include guard），用于避免同一翻译单元重复处理头文件内容。`#pragma once` 是常见替代写法，但不属于 ISO C++ 标准。

Q8：如何用 g++ 编译 C++ 程序并运行生成的可执行文件？

先用 `g++` 编译源文件，并通过 `-o` 指定输出路径。g++ 是 GNU C++ 编译器驱动：按 C++ 规则编译，并在链接阶段自动链接 C++ 标准库（同一份 C++ 源码改用 gcc 驱动时需手动指定标准库），所以 C++ 源码应使用 g++。编译成功后，执行生成文件即可运行程序：

Q9：用 g++ 静态链接 init.cpp 时，各选项做什么，为什么产物通常比动态链接的大？

静态链接会把所需静态库代码并入输出的可执行文件。两种构建都启用 `-g` 时，静态链接产物通常更大。下面的命令示范主机工具链，不保证产物可直接作为目标设备的 `/init`。

Q13：g++ 如何编译并静态链接多个 C++ 源文件？

这条命令把五个 C++ 源文件分别编译，再将它们与所需库链接成 `build/init-second-stage` 可执行文件。

04-language/C++/03-POSIX接口.md（5 题）

Q1：POSIX 接口是什么？C++ 程序何时使用？

POSIX（Portable Operating System Interface，可移植操作系统接口）是一套**操作系统接口标准**，约定程序如何访问文件、进程、线程等系统能力。它为应用开发者提供共同的调用方式，也为系统实现者规定接口契约。它不是一份源码库，也不属于 C++ 标准库。

Q3：C++ 调用 POSIX 函数需要包含什么头文件、链接什么库？

头文件提供函数声明，链接器解析函数实现。具体头文件和库依函数及目标平台而定，没有供所有 POSIX 函数统一使用的 `libposix`。

Q4：wait() 解析？

wait 只等待本进程 fork 创建的子进程，父子关系记录在内核 task_struct 中。wait 阻塞于可中断睡眠，有两个返回出口。子进程终止时返回其 PID 并回收。被捕获信号打断时返回 -1，errno 为 EINTR（Interrupted system call，系统调用被信号中断）。

Q5：pause() 解析?

pause 无条件挂起，唯一返回方式是被信号打断：永远返回 -1 加 EINTR。具体行为取决于到达信号的处置：

Q6：POSIX mkdir() 如何创建目录并处理权限与已存在路径?

`mkdir()` 声明在 `<sys/stat.h>`。成功返回 `0`。失败返回 `-1`，并设置 `errno`。路径以 `/` 开头时为绝对路径，否则相对进程当前工作目录。

04-language/C++/04-函数与类型.md（1 题）

Q24：static_cast 是什么?

`static_cast<T>(表达式)` 是 C++ 的显式类型转换写法：`T` 指定目标类型，括号中是要转换的表达式。它直接执行转换，原变量的类型不会改变。

04-language/C++/05-数组与字符串.md（2 题）

Q6：如何使用 std::strcmp() 比较 C 风格字符串?

`strcmp` 是 string compare 的缩写，C 标准库的字符串比较函数：比较两个以空字符结尾的字符串内容，相等时返回 `0`，不相等时按字典序返回正值或负值，只保证符号、不保证数值。C 程序经 `<string.h>` 声明，C++ 程序经 `<cstring>` 使用 `std::strcmp`。

Q18：std::strtol 如何将字符串按指定进制转换为整数?

`std::strtol` 将字符串开头的数字按指定进制转换为 `long`，并可返回停止解析的位置。它声明在 `<cstdlib>`。

04-language/C++/06-自定义类型.md（2 题）

Q1：如何解析一个 C++ struct 定义?

按语法把一个 struct 定义拆成四部分来读：struct 关键字、类型名、花括号内的成员声明列表、末尾的分号。定义引入一个新类型，随后用类型名声明的变量才是对象。

Q2：结构体与数组如何组织数据？定义结构体类型会创建对象吗?

二者组织数据的方式不同：

01-android（44 题）

01-android/01-architecture/01-系统架构.md（2 题）

Q1：Android 的五层架构分别承担什么职责，每一层都有哪些典型组件?

Android 的五层是按组件职责组织的架构视图，不是五个进程，也不意味着请求必须逐层经过。典型分层从上到下如下：

Q2：Android 中 Native 代码、库和服务如何区分?

Android NDK 文档在应用开发语境中把 C/C++ 代码称为 native code；JNI 文档把 Java/Kotlin 称为 managed code，并说明 JNI 用于两者互调。这里的 Native 指应用中的原生实现，通常编译为设备 ABI 匹配的 `.so` 并加载到进程运行。

01-android/01-architecture/02-系统启动.md（1 题）

Q2：[tags:init] init 进程的入口是什么，是如何拉起的?

ramdisk 的 `/init` 是可执行文件，不是进程。内核创建 PID 1 的 `kernel_init` 内核线程后，通过 `kernel_execve("/init")` 装载它。执行切到 ELF 入口 `_start` 时，同一个 PID 1 才开始运行用户态 init。Bionic 初始化后再调用 C++ `main()`。这是转换已有任务，不是 `fork` 新进程。

01-android/01-architecture/11-SELinux.md（4 题）

Q1：[tags:SELinux] Android SELinux 是什么？它补上了 Android 哪类访问控制缺口?

SELinux（Security-Enhanced Linux，安全增强型 Linux）是 Android 基于 Linux LSM（Linux Security Modules）实现的**强制访问控制**机制。它给进程主体标记域、给受保护对象标记类型，再由内核或相应的用户态对象管理器按 `allow 域 类型:类 权限` 规则检查访问。没有匹配授权时默认拒绝。

Q2：SELinux 的判定机制分哪几步？一次访问是怎么被放行或拒绝的?

判定机制分三步——建立安全上下文、编译并装载规则、逐次判定：策略在构建与启动阶段准备，进程和对象在创建、标记或查询映射时取得上下文。内核或用户态对象管理器再按访问请求执行判定，没有匹配授权即拒绝。

Q4：作为上层应用开发者，SELinux 的实际应用场景都是什么?

应用开发者通常“被 SELinux 约束”而不是配置它：能做的是识别拒绝、改走合规通道，改 sepolicy 不是应用侧的选项。应用无法自行选择域。平台按 seapp_contexts 中的匹配条件确定域，输入不止签名或 targetSdkVersion。

Q6：一条 avc 日志里 pid、uid、name、scontext、tcontext、permissive 各字段分别由谁拼出?

每个字段的拼装者分三层：libselinux 的 `avc_audit` 拼模板与上下文段，servicemanager 注册的审计回调拼业务字段，logcat 行首的 tag 与 pid 来自写入进程自身。

01-android/07-内存与存储/05-分区与挂载.md（12 题）

Q1：Android 分区是什么，如何理解?

Android 分区是设备存储中为特定用途划出的存储区域或逻辑存储空间，用来组织启动文件、系统软件和用户数据等内容。一个设备有多个分区，可以让不同内容分别管理、更新和保护。

Q2：Android 常见分区各有什么作用?

Android 将启动、系统软件、厂商适配和用户数据组织到不同分区。分区名称和是否存在取决于设备配置、首次发布时的 Android 版本及升级方案，不能假定每台设备都有以下所有分区。

Q3：Android 的动态分区是什么?

动态分区是 Android 的一种分区管理方式：在 `super` 物理分区中划分 system、vendor 等**逻辑分区**，并用元数据记录它们占用的空间。系统可以在允许的容量范围内调整这些逻辑分区，而不必重新划分物理分区表。

Q4：Android /system/etc 目录是什么?

/system/etc 是 Android 系统分区的配置文件目录，承担 Unix 传统中 /etc 的角色：init 的 rc 文件、权限 XML、音频策略等出厂配置都在这里。根目录被分区挂载点占满，Android 没有独立的 /etc，配置就随系统分区放在这里。

Q5：Android 的 super 是什么，解决了什么问题?

`super` 是 Android 动态分区方案中的物理分区，用来容纳和管理 system、vendor 等逻辑分区。它让这些分区在同一个容器内分配空间，便于系统升级时调整各自的容量。

Q6：Android 的 super 为什么通常不包含 userdata?

在常见的 Android 动态分区布局中，`super` 容纳 system、vendor 等系统逻辑分区，`userdata` 通常是独立分区。`/data` 是 userdata 上文件系统的挂载点，不是分区名。

Q7：Android 的 scratch 是什么，怎样为 OverlayFS 提供可写空间?

scratch 是 Android 在部分设备上为调试性 OverlayFS 准备的可写工作空间。它解决的问题是：系统分区可能使用只读文件系统，开发时需要临时替换文件，却不能直接向原分区写入。scratch 提供另外一处存储，OverlayFS 再把这处修改与原系统文件合并，让程序仍通过原路径访问。

Q8：adb remount 的原理是什么?

adb remount 请求设备端为可调试系统分区准备可写访问，具体可能是直接重新挂载，也可能在只读文件系统上建立 OverlayFS。它不把 EROFS 本身变为可写格式。

Q9：/dev/block/dm-0 /mnt/scratch f2fs rw 如何解析?

这行表示当时把设备映射节点 dm-0 提供的 F2FS 文件系统以读写方式挂载到 /mnt/scratch。dm-0 是运行时 device-mapper 编号，不是稳定的物理设备身份；仅凭这一行不能判断其最终后端是否在 super 或数据分区。

Q10：Android 的 verity 是什么，为什么启用后可能清理 OverlayFS?

Android 中讨论分区校验时，verity 通常指 **dm-verity**。其中 dm 指 device-mapper（设备映射器），是 Linux 提供的虚拟块设备框架；verity 表示完整性验证。dm-verity 是建立在 device-mapper 上的块设备校验机制，在读取受保护分区的数据时检查其是否与可信的校验信息一致，帮助发现系统分区被篡改或数据损坏。

Q11：U 盘系统升级后为什么可能再次看到旧文件?

升级可能更新系统镜像中的 lower，而独立的持久化 upper 未清理；如果之后再次挂载覆盖层，upper 的同路径旧文件可能遮蔽新 lower。U 盘只是升级包的载体，实际写入和擦除范围由升级方案决定。

Q12：如何确认升级前后的 upper 是否是同一持久化对象?

需要建立跨时间的设备和文件证据链，不能只比较 /mnt/scratch 路径或 dm 编号。记录每次启动的 boot ID、slot、mountinfo、设备映射和动态分区信息，再比较 upper 文件清单与哈希；任一后端重建、路径变化或采集权限不足都可能改变判断。

01-android/05-graphics/01-渲染管线与VSync.md（2 题）

Q1：屏幕显示一帧的全流程?

以 Android 13 普通硬件加速窗口为例，一帧经历应用绘制、buffer 提交、SurfaceFlinger 合成和显示系统呈现。各阶段相互衔接，但不在同一线程或同一时刻完成。

Q2：Android 硬件渲染与软件渲染有何区别，如何选择？

以 Android 13 普通 View 窗口为例，**硬件渲染由 CPU 上的线程组织绘制，再交 GPU 参与生成像素；软件渲染由 CPU 直接计算像素**。两者都会生成窗口 buffer，交给 SurfaceFlinger 合成。

01-android/05-graphics/02-GPU合成与显示.md（2 题）

Q1：设置窗口顶部透明时，为什么看到桌面不代表设置窗口的 buffer 包含桌面像素？

普通硬件加速设置窗口的 buffer 保存该窗口一次绘制的图像。桌面像素通常留在桌面图层的 buffer 中，由 SurfaceFlinger 与设置窗口及系统栏内容合成后显示。

Q2：Android 13 的 WMS 遮挡判断与 SurfaceFlinger 逐帧可见区域为什么可能不同？

WMS 判断 Activity 的目标可见性，SurfaceFlinger 判断当前输出中各图层实际可贡献的区域。前者可在转场动画前决定隐藏下层，后者仍会按每帧图层状态处理。

01-android/13-performance/01-应用/06-图片加载/01-格式与体积.md（7 题）

Q1：位图是什么？

位图用二维像素网格表示图像，每个像素记录颜色及可选的透明度，适合照片和纹理丰富的图像，因为能逐像素保存细节。放大时可能出现明显像素化。位图是一种表示方式，不等同于某种压缩算法。

Q2：矢量图形是什么？

矢量图形用点、线、曲线和路径等几何对象及其填充、描边属性描述画面，适合图标、标志和简单线稿，因为可按目标尺寸绘制并保持边缘清晰。照片若用大量几何对象描述，通常不如位图合适。

Q3：栅格化是什么？

栅格化（rasterization）是把路径、曲线等矢量图形描述映射为目标尺寸上的像素，用于显示或输出的过程。渲染器据图形覆盖范围确定像素颜色和透明度，抗锯齿可平滑边缘。SVG 或 VectorDrawable 可按新的目标尺寸重新栅格化。位图放大只能插值，不能恢复矢量细节。

Q4：JPEG、PNG、GIF 和 SVG 有什么区别？

JPEG、PNG、GIF 按像素表示画面。SVG 用路径和形状描述画面，显示时再转换为像素。它们的压缩方式和适用内容不同：

Q5：Android 应用图片资源有哪些类型，分别适合什么场景？

按表示方式和用途，Android 图片资源分为位图、XML 绘图资源和动画资源：

Q6：图片像素尺寸与文件大小有什么关系？

像素尺寸用宽 × 高表示，口语中常称像素分辨率。它决定像素数量，但不能直接换算压缩文件大小。同尺寸图片的文件大小还取决于内容和编码方式：

Q7：降低 JPEG 质量会怎样影响文件大小、画质和内存？

降低 JPEG 质量通常会减小文件并损失细节，但不会改变解码输出尺寸。相同宽高和像素配置下，Bitmap 像素内存基本相同：

01-android/13-performance/01-应用/06-图片加载/02-解码与内存.md（6 题）

Q1：压缩文件很小，为什么解码后仍可能占大量内存？

压缩文件大小表示编码数据量，不能代表解码后的 Bitmap 内存。像素内存主要由解码输出尺寸和像素配置决定。

Q2：怎样减少 Bitmap 解码临时缓冲区的重复分配？

Android 11 的 BitmapFactory.decodeStream 默认使用约 16 KiB 临时缓冲。频繁解码时，可通过 BitmapFactory.Options.inTempStorage 复用该数组，减少小额重复分配。这不会减少 Bitmap 像素内存。

Q3：Bitmap 复用为什么可能保留较大的分配容量？

大 Bitmap 被复用来存放较小结果时，当前像素量会减少，但底层分配容量可能不变。对象留在缓存或池中时，这部分容量仍被占用。

Q4：Android Studio Memory Profiler 怎样判断 Bitmap 由谁持有？

堆转储能显示 Bitmap 的 Java 包装对象及其引用链。沿引用链可判断它由界面、缓存还是对象池持有。

Q5：如何限制或清理过大的 Bitmap 闲置对象？

为图片缓存和对象池设置字节上限。超限或界面退出时，淘汰可重建且不再使用的 Bitmap。

Q6：Bitmap 像素数据为什么放在 Native Heap，而不是 Java Heap？

Android 8.0（API 26）起，普通软件 Bitmap 的 Java 包装对象在 Java 堆，像素缓冲在 Native Heap。这样可避免大块像素数据占用 Java 堆，但只是改变内存归属，进程总内存仍会增加。

01-android/11-构建系统/02-Soong模块.md（1 题）

Q1：[tags:Soong] AAOS 构建中的 Soong 和 Soong 模块分别是什么？

Soong 是 Android 的构建系统之一。Soong 模块是它构建图中的基本单元。 `Android.bp` 声明模块，Soong 读取声明并解析依赖与构建变体，生成 Ninja 构建规则。Ninja 再执行具体的编译、链接等命令。

01-android/11-构建系统/03-可执行文件.md（5 题）

Q1：静态链接与动态链接有什么区别？

区别在库代码何时并入程序，以及可执行文件运行时依赖什么：

Q2：Android 中的动态链接器是什么，算程序还是库？

是程序，不是库：内核装载动态可执行文件时按 `PT_INTERP` 先装入它并开始执行，由它加载依赖的共享库、解析符号，之后才进入主程序入口。Android 上它是 `/system/bin/linker64`（32 位设备为 `/system/bin/linker`），位于系统分区。

Q5：为什么两种链接产物都能被内核直接执行，文件名为什么没有后缀，执行过程有什么区别？

内核判断能否执行看文件内容与权限位，不看文件名：execve 认出文件头的 ELF 魔数、文件又带执行位，就进入装载，因此两个产物都能直接执行，也都不需要后缀。二者共用的校验相同，区别在内核接手后由谁完成装载。

Q6：ELF 解析？

ELF（Executable and Linkable Format，可执行与可链接格式）是 Linux 与 Android 描述本机二进制的文件格式：可执行程序、共享库 `.so`、可重定位目标文件和核心转储都用它。ELF 只是格式规范，规定字节如何组织。一个文件能否启动，取决于它的 ELF 类型、CPU 架构和依赖，而不是“是不是 ELF”。

Q8：Android 本机可执行文件、ELF、Soong 模块名和设备路径分别是什么？

这里的“本机可执行文件”指可由 Linux 内核按 `exec` 机制启动的本机程序，Android 平台二进制常见为 ELF。普通进程通常通过 `execve` 或 `execveat` 请求内核装载。启动时内核也可用内部的 `kernel_execve` 启动 `/init`。APK 是应用包，不是本机可执行映像。系统创建或复用应用进程后，由 ART 加载并执行其中的 DEX。共享库虽常为 ELF，仍需动态链接器加载，不能独立启动。…

01-android/10-原生层/01-内核与GKI.md（1 题）

Q2：ramdisk、initramfs 和 rootfs 都是什么？内核启动为什么需要它？

ramdisk 是启动时提供给内核的一份最小文件归档。内核将它展开为初始根文件系统（rootfs），再从中启动 `/init`。它解决了系统分区尚未挂载、但挂载所需程序和配置又必须先运行的依赖问题。

01-android/10-原生层/02-驱动运行时.md（1 题）

Q1：应用开发中，常说的串口是什么，与 USB 传输有什么区别，通常用在什么场景？

日常硬件开发里说的“串口”通常指 UART（Universal Asynchronous Receiver/Transmitter，通用异步收发器）：设备通过发送和接收线逐位交换数据。USB 本身也是串行总线，但它有独立的总线协议。这里比较的是 UART 串口与 USB。

08-项目经验（2 题）

08-项目经验/00-项目缺陷画像与复盘.md（2 题）

Q1：某车机项目 85 天开发窗口的提交流呈现什么缺陷画像，哪些数据支持“功能开发在前、提测放量在后”的判断？

画像由三个指标构成：

Q2：某车机项目被修复缺陷中必现占 79.2%、已闭环占 89.5%，这两个比例分别能说明什么、不能说明什么？

口径为某车机项目 85 天窗口内的 543 个被修复缺陷单：

02-os（49 题）

02-os/05-Linux启动/01-Linux启动.md（14 题）

Q1：制作并用 QEMU 启动 initramfs 时，rootfs/ 要准备什么，/init、/dev/console 和 console= 如何配合？

`rootfs/` 是 initramfs 的打包目录。内核解包后，其内容成为运行时根目录 `/`，并执行其中的 `/init` 作为 PID 1。要让用户态和内核日志都能通过 QEMU 串口显示，还需准备控制台设备节点、匹配的内核参数和 QEMU 串口连接。

Q2：Linux 中 std::cout 的输出如何到达 QEMU 宿主机终端？

输出链路从 `std::cout` 写入标准输出开始，经过 `/dev/console`、Linux 控制台和 QEMU 虚拟串口，最后到达宿主终端。要显示日志，init 的标准输出须连接到 `/dev/console`，内核参数须选择对应串口，QEMU 也须把该串口连接到宿主终端。

Q3：为什么 initramfs 要预置 /dev/console，等 /init 挂好 devtmpfs 不行吗？

不能等。devtmpfs 要由 `/init` 运行后执行挂载才会生成节点，而内核在启动 `/init` 前就要打开 `/dev/console`，把它接到早期用户态的标准输入、输出和错误——时序上 devtmpfs 帮不上忙，与本内核是否编译 devtmpfs 无关。节点缺失时，这条控制台通路无法按预期建立。

Q4：如何用 find、cpio 和 gzip 打包并检查 initramfs?

`rootfs/` 是主机上的目录，内核启动时拿到的是 initramfs 镜像数据，不能直接遍历这个目录。因此使用 `cpio` 将文件路径和元数据编码成归档。`newc` 是内核支持的归档格式，能记录权限和设备节点等信息。内核据此还原初始根文件系统。`gzip` 用来减小镜像体积。打包后检查压缩流并列出归档内容，可提前发现损坏或漏文件，不必解包。

Q6：QEMU 在前台运行且终端没有提示符、串口空白，能判断 Linux 内核卡住了吗?

不能。没有提示符只表示宿主 Shell 还在等待前台 QEMU 退出；QEMU 进程存活也不代表客体内核仍在正常执行。串口空白只能说明当前终端没有收到串口字符，无法单独判断是内核没有输出、Linux 没选中串口控制台，还是 QEMU 没把虚拟串口接到终端。

Q7：已看到 earlycon 内核日志，却没有 /init 输出，说明什么?

这只说明早期日志通道已经输出成功，不代表正式的 `ttyS0` 串口驱动已注册。`earlycon` 在正式驱动初始化前直接输出内核早期信息。`/init` 的输出则需要正式串口控制台、`/dev/console` 和 QEMU 串口连接组成完整通路。

Q8：8250 串口数设为 0 时为什么没有 ttyS0，8250.nr_uarts=1 有什么作用?

`CONFIG_SERIAL_8250_RUNTIME_UARTS=0` 让 8250 驱动启动时注册 0 个串口，因此不会提供正式的 `ttyS0`。这是内核驱动的注册数量设置，不代表 QEMU 没有模拟 COM1。

Q9：启动日志 Serial: 8250/16550 driver 和 Run /init as init process 各说明到哪一步?

这两条日志分别标记了串口驱动初始化和内核尝试启动 `/init`，都不能单独证明整条输出链路已经正常。

Q10：Linux 中文件描述符 1 为什么不固定指向屏幕，Mini_Android 的 /init 又如何获得标准输出?

文件描述符 `1` 是进程打开对象清单中的编号，按约定承担标准输出的作用。它指向哪里，由启动程序的一方安排，可以是终端、文件或管道。

Q11：如何创建普通文件和设备节点?

`touch` 可创建空普通文件，shell 重定向可创建并写入内容，设备节点则用 `mknod` 创建。设备节点是文件系统中的特殊文件，不存放普通文件数据，也不等于设备本身。

Q12：mknod、file 和 ls -l 怎么用?

这些命令分别创建设备节点和检查文件。示例中的路径都相对于项目目录：

Q13：chmod 如何使用?

`chmod` 修改文件或目录的权限位，语法是 `chmod 模式 路径`——模式在前、路径在后。给脚本加执行权限最简写法是 `chmod +x 路径`（符号模式，在现有权限上增加执行位），完整指定三位权限用 `chmod 755 路径`（数字模式）。

Q14：mount(proc, /proc, proc, 0, nullptr) 中来源名、挂载点、选 proc 实现的具体目的和作用分别是什么?

`mount()` 是 Linux 的挂载系统调用接口，不是终端里的 `mount` 命令。一次挂载完成的是“把一个文件系统实例接进当前命名空间的一颗目录”：source 与 filesystemtype 决定挂什么，target 决定接在哪。代码把提供进程信息的 procfs 挂到 `/proc`，把提供内核设备信息的 sysfs 挂到 `/sys`。成功返回 `0`，失败返回 `-1` 并设置 `errno`。

Q15：怎么把制作好的 system.img 作为 virtio 磁盘交给 QEMU 启动，命令各参数是什么意思，启动后会发生什么?

QEMU 在模拟一台完整的电脑，每条参数都是往虚拟机上焊硬件或规定行为：`-m` 插内存条，`-kernel` 与 `-initrd` 绕过硬盘引导、直接把内核和初始内存盘交给这台机器，`-append` 写内核启动参数，`-drive` 焊一块盘片为宿主机 system.img 文件的硬盘，其余参数规定输出与退出行为。

02-os/01-文件系统/01-文件与目录.md（5 题）

Q1：块设备是什么?

块设备是以“块”为单位读写的设备文件：数据按固定大小的块组织，每个块可寻址，支持跳到任意块直接访问。磁盘、固态盘、分区和 loop 设备都是块设备。字符设备则以字节流顺序访问，如键盘和串口。两者的区别集中在访问方式、内核路径和系统里的呈现。

Q3：NVMe 是什么？/dev/nvme1n1p4 怎么理解?

NVMe（Non-Volatile Memory Express，非易失性内存主机控制器接口规范）规定主机与存储控制器怎样交换命令和数据。**NVMe 存储**通常指采用 NVMe 规范供主机访问的存储设备或存储资源，例如通过 PCIe 连接的 NVMe 固态硬盘。NVMe 是接口规范，固态硬盘则是实际提供持久化存储的设备。

Q4：PCIe 是什么?

PCIe（Peripheral Component Interconnect Express，外围组件互连express）是主机与设备间的高速串行点对点总线：NVMe 固态盘、显卡、网卡等挂在它上面，经它与 CPU 和内存交换命令与数据。它决定运力，不决定设备语义。

Q12：什么是 loop 设备，它怎么把普通文件变成块设备？

loop 设备是 Linux 提供的虚拟块设备：背后没有物理磁盘，而是把一个普通文件当作“盘体”，让这个文件以块设备的身份被访问。挂载镜像、把镜像交给虚拟机，都是先把镜像文件关联到某个 /dev/loopN，再按块设备使用。

Q23：device-mapper 是什么？Android 如何用它访问动态分区？

device-mapper（设备映射器，简称 dm）是 Linux 内核中的虚拟块设备框架。它根据映射表，在已有块设备之上提供新的块设备；上层读写新设备时，内核按映射规则处理请求，再访问对应的底层设备。

02-os/01-文件系统/02-文件操作模型.md（1 题）

Q1：文件描述符 fd 是什么？

文件描述符（file descriptor，fd）是进程用来引用已打开对象的非负整数，对象可以是普通文件、管道或套接字。内核维护按进程区分的 fd 表；进程把 fd 传给 `read()`、`write()`、`lseek()`、`close()` 等系统调用时，内核在发起调用的进程表中查找对应的打开文件描述。因此，fd 是进程上下文中的编号，不是文件名、文件内容或打开对象本身。

02-os/01-文件系统/05-VFS与挂载.md（5 题）

Q1：文件系统实例是什么？为什么需要挂载才能按路径访问？

文件系统实例是内核管理的一套具体文件系统。例如，ext4 是文件系统类型，某个分区上的 ext4 文件系统是具体实例。**挂载**将实例的根目录接入进程可见的目录树，使应用能从挂载点按路径访问其中的文件。

Q2：VFS 是什么，解决了什么问题？

VFS（Virtual File System，虚拟文件系统）是内核中的文件系统抽象层。它的主要目的是**统一文件访问接口，并让不同文件系统接入同一目录树**：应用使用 open、read、write 等接口，内核再将操作交给对应的文件系统实现。

Q3：通用 Linux 的 fstab 是什么，系统怎样使用它？

fstab 是 file system table（文件系统表）的缩写。通用 Linux 使用 `/etc/fstab` 保存文件系统的挂载配置：每条配置说明一个文件系统从哪里来、挂载到哪个目录，以及如何挂载。这样，系统启动时就可以根据预先写好的配置安排挂载，不必每次都由用户手动指定。

Q4：OverlayFS 是什么？如何合并目录，以及处理读写和删除？

OverlayFS 是 Linux 的联合文件系统：它把一个或多个下层目录与可选的上层目录组合成一个合并视图，应用通过挂载点访问结果。它提供统一的路径视图，不取代存储设备或底层文件系统。

Q5：OverlayFS 中的upper重启后还会生效吗？如何恢复？

可能会。如果通过 `adb remount` 建立了 OverlayFS，再用 `adb push` 替换系统 APK，测试版通常保存在 upper，原 APK 仍在 lower。upper 的后端未被清理且数据已持久化时，重启后文件可能保留；只有对应覆盖层重新挂载，上层文件才会再次遮蔽原 APK。

02-os/01-文件系统/06-文件系统布局.md（2 题）

Q1：F2FS 是什么？

F2FS（Flash-Friendly File System）是 Linux 支持的可读写文件系统，面向基于 NAND 闪存的存储设备设计。它负责组织文件、目录、元数据和数据块，并将读写请求交给底层块设备。

Q2：EROFS 是什么？

EROFS（Enhanced Read-Only File System）是 Linux 的只读镜像文件系统，适合发布后不需原地修改的系统镜像。文件系统内容在构建镜像时组织，运行时可挂载读取；它支持按配置使用压缩等空间优化。

02-os/06-Linux构建/01-镜像编译.md（6 题）

Q1：镜像（image）是什么？

镜像（image）是对某个存储对象的完整字节级副本：将一块磁盘、一个分区、一段内存或一块芯片上的全部内容——包括文件、分区表、空闲空间，乃至已删除但尚未被覆盖的数据残留——逐字节复制到一个文件中。

Q2：归档（archive）是什么？它和 tar、zip、gzip、cpio 是什么关系？

归档是一个概念、一个动作：“把一堆文件打包成一个文件”，每个文件的路径、权限、时间戳都记在包里，到目的地解开还原。

Q3：ISO 是什么？为什么它也算镜像，还能直接拿来装系统？

ISO 是光盘镜像——把一张 CD/DVD 上的全部内容逐扇区复刻成的文件。名字要先说清：ISO 不是英文缩写，是国际标准化组织（International Organization for Standardization）从希腊语 isos（相等）选定的定名。它发布的 ISO 9660 就是光盘上用的文件系统格式，.iso 扩展名由此而来。Ubuntu 安装镜像、Windows 安装镜像都是它。

Q4：Ubuntu 的 ISO 是怎么一步步装出系统的？

全流程五步（U 盘安装）：

Q5：一个独立的 ext4 system 镜像（如 Android 的 system.img）解析？

ext4（fourth extended filesystem，第四代扩展文件系统）是 Linux 的文件系统格式，Android 系统分区用的就是它。ext4 system 镜像，就是把一块装好 ext4 的分区整个拷贝进一个普通文件。

Q6：mkfs.ext4 全解析

`mkfs.ext4` 在块设备或镜像文件上创建 ext4 文件系统，调用形式为 `mkfs.ext4 [选项] 设备 [块数]`。设备必须是已存在的分区、loop 设备或普通镜像文件。省略块数时文件系统使用整个设备。它是 e2fsprogs 里 mke2fs 的别名，`mke2fs -t ext4 设备` 与 `mkfs.ext4 设备` 等价，调用名只决定默认文件系统类型。

02-os/03-processes/01-进程与调度.md（1 题）

Q1：内核有“进程”这个概念吗？内核为什么能运行？

进程是内核管理的任务，不是内核运行的前提。内核启动初期由 CPU 直接执行特权代码。完成基础初始化后，才创建并调度内核线程和用户进程。

02-os/03-processes/03-套接字接口.md（11 题）

Q1：套接字和套接字描述符分别是什么？由谁维护，数量由什么决定？

套接字是内核维护的通信端点，套接字描述符是进程访问它的整数 fd。对象与编号分属内核和进程两侧，数量随创建与关闭动态变化，受 fd 上限、系统文件表和内存限制。

Q2：socket 函数的声明、参数和返回值怎样读？

Q3：SOCK_DGRAM 和 SOCK_STREAM 的区别？

两者的核心区别是通信是否先建立逻辑连接，以及应用接收的是独立报文还是连续字节流。

Q4：SOCK_SEQPACKET 和 SOCK_RAW 分别提供什么服务？

SOCK_SEQPACKET 关注有连接的记录交付，SOCK_RAW 允许应用接触较低层协议数据。它们都是套接字类型，但各自支持范围由通信域与协议决定。

Q5：套接字什么时候保留连接或直接 close，何时才需要 shutdown？

如果这条连接以后既不发送、不接收，也不复用，直接 close(fd)，不需要先 shutdown。shutdown 主要用于还不能关闭整个连接，却要先结束一个收发方向的情况，例如结束发送后继续接收响应。close 释放当前 fd 引用，shutdown 改变套接字的收发方向但不释放 fd，最终仍需 close。

Q6：接受文件描述符的函数，为什么不一定能用于套接字？

文件描述符统一了访问入口，没有统一所有对象的能力。判断某个函数能否用于套接字，要看它要求底层对象提供什么语义。

Q7：套接字的地址结构解析？

套接字地址包含**地址族**和该族的**定位信息**。地址族决定后面的字段怎样解释，定位信息指出具体端点。程序按地址族创建实际结构，以下代码只摘录关键字段，省略了实现可能添加的填充字段。

Q8：gethostent 等枚举接口如何管理遍历状态和返回结果？

这组函数用于**逐条遍历主机记录**。这里的“主机记录库”不是固定的一份系统文件，而是主机信息的逻辑集合。每条记录描述一个主机名及其 IP 地址。

Q9：主机、网络、协议和服务数据库分别查什么？

Q10 讲的是**怎样逐条遍历主机记录**，包括 sethostent、gethostent、endhostent 和 hostent 的返回内容。Q19 讲的是**四类记录各自查什么、按什么条件查、何时使用**。这些接口只查询名称或编号资料，不创建套接字、不连接远端，也不证明相应服务正在运行。

Q10：如何将套接字与地址关联？

bind 的作用是把本地地址关联到已创建的套接字。下面把服务器和客户端的核心代码分散到对应解释中；代码片段省略头文件和错误处理。

Q11：面向连接的套接字如何建立连接？

面向连接的套接字（`SOCK_STREAM` 或 `SOCK_SEQPACKET`）要先建立两端关系，之后才能交换数据。客户端用 `connect` 发起连接；服务器用监听套接字接入请求，再由 `accept` 取出连接并返回专门收发数据的新套接字。

02-os/04-virtualization/01-虚拟化.md（3 题）

Q1：虚拟机和虚拟机监控程序有什么区别，VMM 的两种类型如何划分？

虚拟机（VM，Virtual Machine）是由虚拟 CPU、内存和设备组成的运行环境，可以在其中运行客体操作系统及应用。虚拟机监控程序（VMM，Virtual Machine Monitor）是创建和运行虚拟机的管理软件，负责协调虚拟资源、执行客体并处理或委托设备访问。

虚拟机是客体使用的环境，VMM 是提供和管理这个环境的组件。

Q2：hypervisor 是什么，与 VMM 有什么区别?

hypervisor（虚拟机监控程序）是管理虚拟机运行的虚拟化层，负责让多个客体系统共享物理资源，并约束它们对 CPU、内存和设备的访问。它提供客体运行所需的隔离与仲裁机制，让每个客体看到分配给自己的虚拟机器。

Q3：车机为什么会同时运行 Linux 和 Android，怎样判断双系统是否采用虚拟化?

采用虚拟化的动机通常是让两套需要独立内核的系统复用一套硬件，同时明确分配 CPU、内存和设备。例如产品可以让 Linux 承载一类设备服务、Android 承载交互界面，但具体分工取决于产品。看到两者并存只能说明有两个软件环境，必须再确认它们是否同时运行、是否共用内核与处理器资源，才能判断是否存在虚拟机。

02-os/00-基础/01-操作系统基础.md（1 题）

Q2：32 位整数的大小端如何影响内存字节排列和跨主机解释?

字节序改变多字节整数在内存中的排列，不改变整数的数值。以值 0x04030201 为例，最高有效字节是 0x04，最低有效字节是 0x01。假设字符指针 cp 指向该整数的最低内存地址，从低地址到高地址读取四个字节：